

```
timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
```

```
module CSR_tb();
```

```
    // Clock and Reset
```

```
    reg i_clk;
```

```
    reg i_rst_n;
```

```
    // Inputs
```

```
    reg i_external_interrupt;
```

```
    reg i_software_interrupt;
```

```
    reg i_is_inst_illegal;
```

```
    reg i_is_ecall;
```

```
    reg i_is_ebreak;
```

```
    reg i_is_mret;
```

```
    reg [6:0] i_opcode;
```

```
    reg [4:0] i_func5;
```

```
    reg [31:0] i_y;
```

```
    reg [2:0] i_func3;
```

```
    reg [11:0] i_csr_index;
```

```
    reg [31:0] i_imm;
```

```
    reg [31:0] i_rs1;
```

```
    reg [31:0] i_pc;
```

```
    reg writeback_change_pc;
```

```
    reg i_minstret_inc;
```

```
    // Outputs
```

```
    wire [31:0] o_csr_out;
```

```
    wire [31:0] o_return_address;
```

```
    wire [31:0] o_trap_address;
```

```
    wire o_go_to_trap_q;
```

```
    wire o_return_from_trap_q;
```

```
    // Instantiate CSR Module
```

```
    CSR uut (
```

```
        .i_clk(i_clk),
```

```
        .i_rst_n(i_rst_n),
```

```
        .i_external_interrupt(i_external_interrupt),
```

```
        .i_software_interrupt(i_software_interrupt),
```

```
        .i_is_inst_illegal(i_is_inst_illegal),
```

```
        .i_is_ecall(i_is_ecall),
```

```
        .i_is_ebreak(i_is_ebreak),
```

```
        .i_is_mret(i_is_mret),
```

```
        .i_opcode(i_opcode),
```

```

.i_funct5(i_funct5),
.i_y(i_y),
.i_funct3(i_funct3),
.i_csr_index(i_csr_index),
.i_imm(i_imm),
.i_rs1(i_rs1),
.o_csr_out(o_csr_out),
.i_pc(i_pc),
.writeback_change_pc(writeback_change_pc),
.o_return_address(o_return_address),
.o_trap_address(o_trap_address),
.o_go_to_trap_q(o_go_to_trap_q),
.o_return_from_trap_q(o_return_from_trap_q),
.i_minstret_inc(i_minstret_inc)
);

```

```

// Clock Generation

```

```

always #5 i_clk = ~i_clk;

```

```

// Test Sequence

```

```

initial begin

```

```

    $dumpfile("CSR_tb.vcd");

```

```

    $dumpvars(0, CSR_tb);

```

```

    // Initialize Inputs

```

```

    i_clk = 0;

```

```

    i_rst_n = 0;

```

```

    i_external_interrupt = 0;

```

```

    i_software_interrupt = 0;

```

```

    i_is_inst_illegal = 0;

```

```

    i_is_ecall = 0;

```

```

    i_is_ebreak = 0;

```

```

    i_is_mret = 0;

```

```

    i_opcode = 0;

```

```

    i_funct5 = 0;

```

```

    i_y = 0;

```

```

    i_funct3 = 0;

```

```

    i_csr_index = 0;

```

```

    i_imm = 0;

```

```

    i_rs1 = 0;

```

```

    i_pc = 32'h00000000;

```

```

    writeback_change_pc = 0;

```

```

    i_minstret_inc = 0;

```

```

    // Apply Reset

```

```

    #10 i_rst_n = 1;

```

```

$display("RESET DONE");

// ---- Test Standard CSR Write ----
i_opcode = `CSR;
i_funct3 = `CSRRW;
i_csr_index = `MSTATUS;
i_rs1 = 32'h12345678;
#10;
$display("CSRRW Test: MSTATUS = %h", o_csr_out);

// ---- Test CSR Read ----
i_funct3 = `CSRRS;
i_rs1 = 32'h00000000;
#10;
$display("CSRRS Test: MSTATUS = %h", o_csr_out);

// ---- Test Atomic ADD on CSR ----
i_opcode = `AMO;
i_funct5 = `AMO_ADD;
i_csr_index = `MSTATUS;
i_rs1 = 32'h00000001;
#10;
$display("AMO_ADD Test: MSTATUS = %h", o_csr_out);

// ---- Test Atomic XOR on CSR ----
i_funct5 = `AMO_XOR;
i_rs1 = 32'hAAAAAAAA;
#10;
$display("AMO_XOR Test: MSTATUS = %h", o_csr_out);

// ---- Test Atomic AND on CSR ----
i_funct5 = `AMO_AND;
i_rs1 = 32'hFFFFFFFF0;
#10;
$display("AMO_AND Test: MSTATUS = %h", o_csr_out);

// ---- Test Atomic OR on CSR ----
i_funct5 = `AMO_OR;
i_rs1 = 32'h0000000F;
#10;
$display("AMO_OR Test: MSTATUS = %h", o_csr_out);

// ---- Test Atomic SWAP on CSR ----
i_funct5 = `AMO_SWAP;
i_rs1 = 32'hDEADBEEF;
#10;

```

```

$display("AMO_SWAP Test: MSTATUS = %h", o_csr_out);

// ---- Test Exception Handling (ECALL) ----
i_opcode = `CSR;
i_is_ecall = 1;
#10;
i_is_ecall = 0;
$display("ECALL Test: Trap Address = %h", o_trap_address);

// ---- Test Exception Handling (EBREAK) ----
i_is_ebreak = 1;
#10;
i_is_ebreak = 0;
$display("EBREAK Test: Trap Address = %h", o_trap_address);

// ---- Test MRET (Return from Trap) ----
i_is_mret = 1;
#10;
i_is_mret = 0;
$display("MRET Test: Return Address = %h", o_return_address);

// End Simulation
#20;
$finish;

end

endmodule

```